



EJERCICIO MACHINE LEARNING PARTE 1

Alejandro González Fernández de Castro

Ejercicio Machine Learning Parte 1

Alejandro González
2025-08-18

Índice

- 0. Introducción 2
- 1. Exploración y transformación de los datos..... 2
- 2. Árbol de decisión 4
 - 2.1 Búsqueda paramétrica y evaluación de los árboles candidatos 4
 - 2.2 Reglas del árbol10
 - 2.4 Visualización del árbol.....11
 - 2.4 Importancia de las variables12
- 3. Mejor búsqueda paramétrica por Random Forest y XGBoost.....13
 - 3.1 Búsqueda paramétrica por Random Forest.....13
 - 3.2 Optimización con XGBoost15
- 4. Comparativa de los modelos:17

0. Introducción

Este documento se ha realizado con RMarkdown y Rstudio, utilizando el intérprete para Python Reticulate. En general se muestran todos los códigos salvo los redundantes.

El enunciado nos pide desarrollar un modelo predictivo que nos permita saber si los coches de segunda mano, en base a sus características, hay que pintarlos de blanco o no. La variable dependiente "Color" tomará los valores White o Black.

El código de lectura y carga de las librerías no se muestra, puesto que es análogo al de los cuadernos del master.

1. Exploración y transformación de los datos

Arrancamos, nunca mejor dicho, con la exploración preliminar de las variables. El dataset no tiene missing values a menos que se eliminen outliers de las variables, pero sí hay filas posiblemente duplicadas según el siguiente código. De acuerdo con lo discutido en la tutoría, se admite que pueden ser modelos duplicados en la fábrica, que pueden ser interesantes, y no se quitarán los duplicados. Por otra parte, la variable respuesta presenta una relación relativamente aceptable de ambos valores, blanco y negro, algo desbalanceada.

```
na_counts = cars.isnull().sum()
print(na_counts[na_counts > 0])
## Series([], dtype: int64)
print(cars.duplicated().any())
## True
cars["Color"].value_counts()
## Color
## Black    2327
## White    2013
## Name: count, dtype: int64
```

El siguiente paso seguido es la verificación de los tipos de variables. Mileage, Levy, Engine volume son variables cuantitativas que están mal interpretadas porque tienen caracteres introducidos en los datos que se deberán eliminar antes de transformar. Cylinders se puede considerar más categórica que numérica y se transformará también.

```
print(cars.head(4).to_string(index=False))

## Price Levy  Manufacturer  Prod. year  Category  Leather interior  Fuel type  Engine volume
Mileage  Cylinders Gear box type Drive wheels  Wheel Color  Airbags
## 39493  891    HYUNDAI      2016      Jeep      Yes      Diesel      2 160931
km        4      Automatic    Front Left wheel White      4
## 1803  761    TOYOTA      2010  Hatchback      Yes      Hybrid      1.8 258909
km        4      Automatic    Front Left wheel White      12
## 1098  394    TOYOTA      2014      Sedan      Yes      Hybrid      2.5 398069
km        4      Automatic    Front Left wheel Black      12
## 941 1053  MERCEDES-BENZ      2014      Sedan      Yes      Diesel      3.5 184467
km        6      Automatic    Rear Left wheel White      12

cars.dtypes
## Price          int64
## Levy          object
## Manufacturer   object
## Prod. year     int64
## Category       object
## Leather interior object
## Fuel type      object
```

```

## Engine volume      object
## Mileage            object
## Cylinders          int64
## Gear box type      object
## Drive wheels       object
## Wheel              object
## Color              object
## Airbags            int64
## dtype: object
cars["Mileage"] = cars["Mileage"].str.replace(' km', ' ', regex=True).astype(int)
##Retiramos los caracteres y la convertimos en numérica
cars["Levy"] = pd.to_numeric(cars["Levy"], errors="coerce")
#Con Coerce se asignan Na.
cars["Engine volume"] = cars["Engine volume"].str.extract(r"([\d.]+)").astype(float) #Eliminación de la característica turbo y la pasamos a numérica
cars["Cylinders"] = cars["Cylinders"].astype(object)
#Conversión de Cylinders a categórica
cars.dtypes

## Price              int64
## Levy              float64
## Manufacturer       object
## Prod. year         int64
## Category           object
## Leather interior   object
## Fuel type          object
## Engine volume      float64
## Mileage            int64
## Cylinders          object
## Gear box type      object
## Drive wheels       object
## Wheel              object
## Color              object
## Airbags            int64
## dtype: object

```

Con la función `describe()` se obtiene directamente los descriptivos básicos de las variables numéricas, permitiendo la primera aproximación a la detección de valores extremos o atípicos. De hecho, se pueden encontrar valores extremos máximos en Price, Levy, Prod.year y sobre todo, Mileage. Valores mínimos extremos hay seguramente en Prod.year, donde hay un coche de 1940.

```

cars.describe().T
##          count      mean      std      min      25%      50% \
## Price      4340.0    19161.052765  2.399052e+04      3.0    1725.0    14113.0
## Levy       3696.0     981.525974  5.530042e+02     87.0     640.0     836.0
## Prod. year  4340.0    2012.849770  3.596240e+00    1943.0    2011.0    2013.0
## Engine volume  4340.0      2.614747  1.013176e+00      0.0        1.8        2.4
## Mileage     4340.0   396052.539401  1.686423e+07      0.0   69918.0   127515.0
## Airbags     4340.0       7.884101  4.600287e+00      0.0        4.0       10.0
##
##          75%      max
## Price      27065.0  6.272200e+05
## Levy       1172.0  1.171400e+04
## Prod. year  2015.0  2.020000e+03
## Engine volume  3.5  6.300000e+00
## Mileage    186542.0  1.111111e+09
## Airbags     12.0  1.600000e+01

```

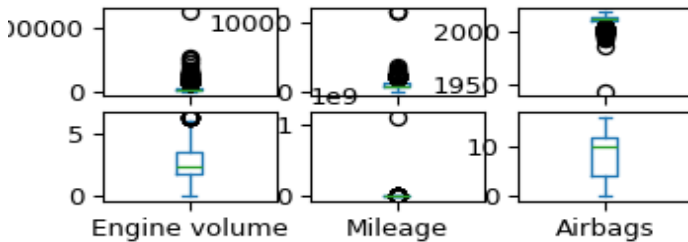
Se puede constatar igualmente con un boxplot:

```

cars.plot(
    kind="box",
    subplots=True,
    layout=(4,3),
    figsize=(4,3),
    sharex=False,

```

```
sharey=False
)
plt.show()
```



Se había considerado eliminar todos aquellos valores extremos que no estén en el rango intercuartílico y verificarlo después con el mismo boxplot la mejora de las distribuciones, con el siguiente Código, pero, como se vio en la tutoría, al tener que considerar outliers en las predicciones, estas se dejan. Como a priori además las distribuciones no parecen alejarse mucho de la normalidad, no consideraríamos otras transformaciones.

Para las variables cualitativas, se agrupan categorías con pocas ocurrencias en Cylinders. Se podría considerar realizar el mismo proceso con otras variables, pero puede perderse información comercial importante, como la marca del coche, etc. Se consideró también eliminar la variable Wheel.

```
cars["Cylinders"] = cars["Cylinders"].apply(lambda x: 4 if x == 4 else "Otro")
```

Finalmente, se preparan los datos de test y train, eliminando la variable respuesta de los predictores, y verificando que las proporciones de la variable respuesta sean similares en ambos:

```
categoricas = cars.select_dtypes(exclude=["number"]).columns.tolist()
categoricas = [x for x in categoricas if x not in ["Color"]]
cars = pd.get_dummies(cars, columns=categoricas, drop_first=True)
X=cars.drop(columns=["Color"])
y= cars["Color"]
X_train , X_test , y_train , y_test = train_test_split(X, y, test_size =0.2, random_state =1234)
print(f'Frecuencia en train: \n{y_train.value_counts(normalize=True)}')
## Frecuencia en train:
## Color
## Black    0.532258
## White    0.467742
## Name: proportion, dtype: float64
print(f'\nFrecuencia en test: \n{y_test.value_counts(normalize=True)}')
## Frecuencia en test:
## Color
## Black    0.551843
## White    0.448157
## Name: proportion, dtype: float64
```

2. Árbol de decisión

2.1 Búsqueda paramétrica y evaluación de los árboles candidatos

Esta sección recoge solo una parte de los resultados de pruebas de malla de hiperparámetros que se han realizado. Se muestra una selección de tres casos explorados. Las primeras pruebas de

búsqueda se realizaron con una profundidad de campo mucho mayor (20), tamaños de hojas pequeños (hasta 3), y tamaños de nodo también igualmente pequeños (15). Invariablemente se obtenían árboles muy profundos, (depths=19-20) con tamaños mínimos de hoja y con diferencias entre el entrenamiento (con un accuracy máximo sobre el dataset de entrenamiento de 0.7) y la validación de 0.17, aproximadamente, que indicaban un sobreajuste significativo. Las métricas basadas en los cuatro segmentos de validación cruzada igualmente daban unos valores de precisión no mayores en general de 0.7. Esta primera tanda de árboles tenía como resultado interesante, de cara al ejercicio, mucha diferencia en la estabilidad de las métricas de bondad según validación cruzada, como se podía observar en los diagramas de caja. Lo que se presenta en esta sección por lo tanto es la segunda fase donde se ha limitado deliberadamente tanto la profundidad de los árboles como el tamaño de los nodos a fin de evitar el sobreajuste. Se han ido probando varias combinaciones en función hasta donde el tiempo de computación de la búsqueda ha permitido. El resultado son tres fases: una primera fase de búsqueda amplia con pocos hiperparámetros y dos búsquedas ajustadas de acuerdo a los primeros resultados: experimentando con el parámetro del tamaño de la hoja y sin él.

El primera paso entonces es la búsqueda paramétrica sencilla con 'gridSearchCV()':

```
params0 = {
    'max_depth': [3, 5, 10],
    'min_samples_leaf': [15, 30, 50],
    'min_samples_split': [30, 50, 75, 100],
    'criterion': ["gini", "entropy"]
}
scoring_metrics = ['accuracy', 'precision_macro', 'recall_macro', 'f1_macro']
grid_search0 = GridSearchCV(estimator=DecisionTreeClassifier(),
                             param_grid=params0,
                             cv=4, scoring = scoring_metrics, refit='accuracy')
grid_search0.fit(X_train, y_train)
## GridSearchCV(cv=4, estimator=DecisionTreeClassifier(),
##              param_grid={'criterion': ['gini', 'entropy'],
##                            'max_depth': [3, 5, 10],
##                            'min_samples_leaf': [15, 30, 50],
##                            'min_samples_split': [30, 50, 75, 100]},
##              refit='accuracy',
##              scoring=['accuracy', 'precision_macro', 'recall_macro',
##                      'f1_macro'])
```

Los resultados se almacenan en una tabla en orden descendente según el criterio de accuracy o precisión, para detectar arriba del todo y rápidamente los modelos candidatos según su puntuación. Se abrevian los nombres de las columnas. El objeto results_abrev_5 permite visualizar los resultados de los árboles de 5 en 5 para ver cuánto decrecen las métricas: a partir del árbol 5 de la parrilla de opciones paramétricas ya la precisión o accuracy, así como el resto de las medidas de bondad, decae por debajo de 0.7; Las cuatro medidas estimadas se comportan de manera consistente, lo que facilita la elección del árbol por las cuatro a la vez.

Los 5 primeros árboles se manejan en una horquilla de profundidad de 10, y es un parámetro que no voy a afinar más, aunque se alcance el extremo de la búsqueda paramétrica que se realizó. Por riesgo de sobreajuste del árbol, lo dejaremos así. La misma razón vamos a aplicar con el tamaño de hoja y los splits. En el primer caso ya el tamaño de hoja (min_samples_leaf) ya es demasiado pequeño en el extremo inferior, por lo que no se debería explorar hacia valores más pequeños. Lo mismo ocurre con el tamaño del nodo.

Por otro lado, los resultados indican mucha consistencia en el resto de las métricas del árbol. Los cuatro métodos de validación parecen fuertemente correlacionados, por lo que no habrá que

solventar discrepancias significativas en cuanto a esas métricas. Es decir, no se necesita valorar 4 árboles muy diferentes según cuatro métricas, sino valorarlo en global.

```
results0 =
pd.DataFrame(grid_search0.cv_results_).sort_values(by="mean_test_accuracy",ascending=False).reset_in
dex(drop=True)
params_df0 = pd.json_normalize(results0["params"])
results0_expanded = pd.concat([params_df0, results0[['mean_test_accuracy',
'mean_test_precision_macro', 'mean_test_recall_macro', 'mean_test_f1_macro']]],axis=1)
results_abrev0 = results0_expanded.rename(columns={
    "criterion": "criterion",
    "max_depth": "depth",
    "min_samples_leaf": "leaf",
    "min_samples_split": "split",
    "mean_test_accuracy": "accu",
    "mean_test_precision_macro": "precision",
    "mean_test_recall_macro": "recall",
    "mean_test_f1_macro": "f1"
})
results_abrev0 = results_abrev0.round(3)
pd.set_option("display.max_rows", None)
pd.set_option("display.max_columns", None)
results_abrev0_5 = results_abrev0.iloc[:5].head(25)
print(results_abrev0_5.head(7))
##      criterion  depth  leaf  split  accu  precision  recall    f1
## 0      entropy    10    15    30  0.718    0.718    0.716  0.716
## 5      entropy    10    30    30  0.692    0.691    0.691  0.691
## 10     entropy    10    15   100  0.687    0.688    0.688  0.686
## 15         gini    10    50    50  0.682    0.683    0.683  0.682
## 20     entropy    10    50    75  0.679    0.680    0.679  0.678
## 25         gini     5    30    75  0.676    0.675    0.675  0.675
## 30         gini     5    30   100  0.674    0.674    0.674  0.673
print(results_abrev0.head(15))

##      criterion  depth  leaf  split  accu  precision  recall    f1
## 0      entropy    10    15    30  0.718    0.718    0.716  0.716
## 1         gini    10    15    30  0.715    0.714    0.714  0.714
## 2      entropy    10    15    50  0.712    0.714    0.710  0.710
## 3         gini    10    15    50  0.707    0.706    0.706  0.705
## 4      entropy    10    15    75  0.694    0.694    0.693  0.693
## 5      entropy    10    30    30  0.692    0.691    0.691  0.691
## 6      entropy    10    30    50  0.692    0.691    0.691  0.690
## 7         gini    10    15   100  0.691    0.692    0.692  0.690
## 8         gini    10    15    75  0.690    0.691    0.690  0.689
## 9      entropy    10    30    75  0.688    0.687    0.687  0.687
## 10     entropy    10    15   100  0.687    0.688    0.688  0.686
## 11         gini    10    50    75  0.684    0.685    0.685  0.684
## 12     entropy    10    30   100  0.683    0.683    0.682  0.682
## 13         gini    10    30    50  0.682    0.682    0.681  0.681
## 14         gini    10    30    30  0.682    0.682    0.681  0.681
```

La búsqueda automática nos daría los siguientes parámetros:

```
print(grid_search0.best_estimator_)
## DecisionTreeClassifier(criterion='entropy', max_depth=10, min_samples_leaf=15,
##                        min_samples_split=30)
```

Así pues, se van a ajustar las búsquedas con los siguientes parámetros de manera más fina:

- Máxima profundidad entre 6 y 10;
- Tamaño mínimo desde 25 hasta 60. Aunque podría llegar hasta 75 según los primeros modelos y en la tutoría se plantean hasta 100 el tiempo de computación me lo hace inviable. Por el extremo inferior sí se fuerza un poco el tamaño del nodo, pensando en que con la profundidad se puede controlar el sobreajuste.

- Tamaño de la hoja de 25 a 40; con tamaños razonablemente grandes se puede controlar el sobreajuste.
- Validación cruzada en 4 segmentos.

```

params = {
    'max_depth': list(range(6, 10)),
    'min_samples_leaf': list(range(25, 40)),
    'min_samples_split': list(range(25, 75)),
    'criterion': ["gini", "entropy"]
}
scoring_metrics = ['accuracy', 'precision_macro', 'recall_macro', 'f1_macro']
grid_search = GridSearchCV(estimator=DecisionTreeClassifier(),
                           param_grid=params,
                           cv=4, scoring = scoring_metrics, refit='accuracy')
grid_search.fit(X_train, y_train)
## GridSearchCV(cv=4, estimator=DecisionTreeClassifier(),
##              param_grid={'criterion': ['gini', 'entropy'],
##                          'max_depth': [6, 7, 8, 9],
##                          'min_samples_leaf': [25, 26, 27, 28, 29, 30, 31, 32,
##                                              33, 34, 35, 36, 37, 38, 39],
##                          'min_samples_split': [25, 26, 27, 28, 29, 30, 31, 32,
##                                              33, 34, 35, 36, 37, 38, 39, 40,
##                                              41, 42, 43, 44, 45, 46, 47, 48,
##                                              49, 50, 51, 52, 53, 54, ...]}],
##              refit='accuracy',
##              scoring=['accuracy', 'precision_macro', 'recall_macro',
##                      'f1_macro'])

```

Aunque no lo tendremos en cuenta, el modelo automático que podemos solicitar a Python es el siguiente:

```

print(grid_search.best_estimator_)
## DecisionTreeClassifier(criterion='entropy', max_depth=8, min_samples_leaf=27,
##                        min_samples_split=43)

```

Los nuevos resultados que se ordenan por el criterio de accuracy indican lo siguiente en la siguiente tabla abreviada.

- Disminuye la optimización mediante validación cruzada cuando se realiza un ajuste más fino de los hiperparámetros. Particularmente parece que se estabiliza en una profundidad del árbol algo más baja (8), algo forzado porque se eliminan los tamaños de hoja más pequeños.
- El parámetro del tamaño de hoja condiciona totalmente el árbol que se obtiene. Todos los árboles que puntúan mejor con los cuatro criterios tienen el mismo tamaño y además tienen las mismas puntuaciones en los criterios de bondad. Únicamente cambia el tamaño del nodo.

##	criterion	depth	leaf	split	accu	precision	recall	f1
## 0	entropy	8	27	43	0.705	0.707	0.706	0.704
## 5	entropy	8	27	31	0.704	0.705	0.705	0.703
## 10	entropy	8	27	44	0.704	0.705	0.705	0.703
## 15	entropy	8	27	49	0.704	0.705	0.705	0.703
## 20	entropy	8	27	52	0.704	0.705	0.705	0.703
## 25	entropy	8	27	42	0.704	0.705	0.705	0.703
## 30	entropy	8	25	26	0.703	0.705	0.704	0.702

##	criterion	depth	leaf	split	accu	precision	recall	f1
## 0	entropy	8	27	43	0.705	0.707	0.706	0.704
## 1	entropy	8	25	45	0.705	0.707	0.706	0.704
## 2	entropy	8	27	27	0.704	0.706	0.705	0.703

## 3	entropy	8	27	47	0.704	0.706	0.705	0.703
## 4	entropy	8	27	55	0.704	0.706	0.705	0.703
## 5	entropy	8	27	31	0.704	0.705	0.705	0.703
## 6	entropy	8	27	26	0.704	0.705	0.705	0.703
## 7	entropy	8	27	30	0.704	0.705	0.705	0.703
## 8	entropy	8	27	41	0.704	0.705	0.705	0.703
## 9	entropy	8	27	46	0.704	0.705	0.705	0.703
## 10	entropy	8	27	44	0.704	0.705	0.705	0.703
## 11	entropy	8	27	28	0.704	0.705	0.705	0.703
## 12	entropy	8	27	29	0.704	0.705	0.705	0.703
## 13	entropy	8	27	39	0.704	0.705	0.705	0.703
## 14	entropy	8	27	50	0.704	0.705	0.705	0.703

Por lo tanto se repite la búsqueda sin considerar el tamaño mínimo de hoja para ver qué resultados diferentes se pueden obtener, y o al menos comprobar si los árboles son más variables y permiten jugar algo más con lo que se pide en el ejercicio.

```
params = {
    'max_depth': list(range(6, 10)),
    # 'min_samples_leaf': list(range(25, 40)),
    'min_samples_split': list(range(25, 75)),
    'criterion': ["gini", "entropy"]
}
scoring_metrics = ['accuracy', 'precision_macro', 'recall_macro', 'f1_macro']
grid_search2 = GridSearchCV(estimator=DecisionTreeClassifier(),
                             param_grid=params,
                             cv=4, scoring = scoring_metrics, refit='accuracy')
grid_search2.fit(X_train, y_train)
```

El modelo automático que podemos solicitar a Python es el siguiente:

```
print(grid_search2.best_estimator_)

## DecisionTreeClassifier(criterion='entropy', max_depth=9, min_samples_split=25)
```

Los nuevos resultados sin tener en cuenta el parámetro indican lo siguiente:

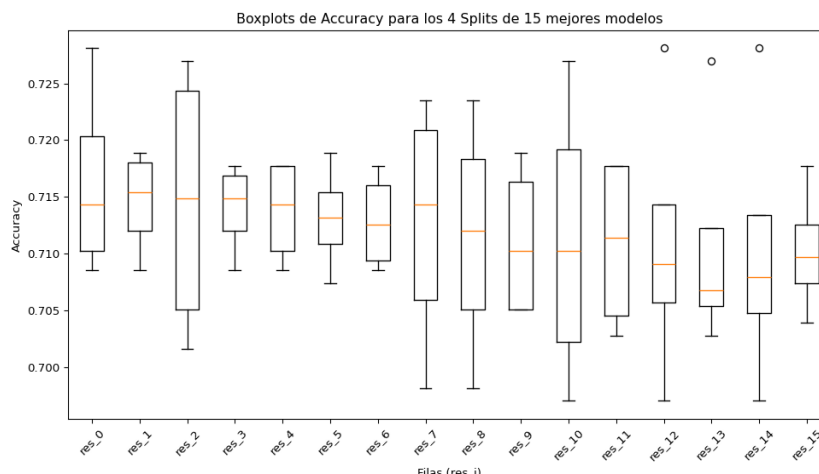
- Aumenta ligeramente la optimización mediante validación cruzada, que se puede ver gracias a un ajuste más fino de los hiperparámetros. Particularmente parece que se estabiliza en una profundidad del árbol más alta (9).
- Se mantiene la consistencia entre las cuatro medidas de bondad de ajuste del árbol; se puede seguir evaluando de manera consistente entre árboles.
- También en general hay muchos más árboles con un accuracy superior a 0.7 con validación cruzada.
- Las diferencias en precisión entre los primeros árboles sigue siendo mínima, pero no obstante ya encontramos variabilidad. Como estos resultados parecen más variables, son más interesantes para proseguir con el ejercicio y realizar la selección del árbol. Respecto a la selección por las cuatro métricas, al ser tan consistentes, se puede uno guiar por las cuatro al mismo tiempo.

##	criterion	depth	split	accu	precision	recall	f1
## 0	entropy	9	25	0.716	0.716	0.716	0.715
## 5	entropy	9	28	0.713	0.712	0.713	0.712
## 10	entropy	9	41	0.711	0.710	0.711	0.710
## 15	entropy	8	28	0.710	0.711	0.711	0.710
## 20	entropy	9	50	0.709	0.708	0.709	0.708
## 25	entropy	9	39	0.708	0.707	0.708	0.707
## 30	entropy	8	31	0.708	0.707	0.708	0.707

##	criterion	depth	split	accu	precision	recall	f1
## 0	entropy	9	25	0.716	0.716	0.716	0.715
## 1	entropy	9	27	0.715	0.714	0.714	0.714
## 2	gini	9	25	0.715	0.714	0.714	0.713
## 3	entropy	9	29	0.714	0.714	0.714	0.713
## 4	entropy	9	26	0.714	0.713	0.713	0.713
## 5	entropy	9	28	0.713	0.712	0.713	0.712
## 6	entropy	9	30	0.713	0.713	0.713	0.712
## 7	gini	9	27	0.713	0.712	0.712	0.711
## 8	gini	9	26	0.711	0.711	0.711	0.710
## 9	gini	9	28	0.711	0.711	0.710	0.710
## 10	entropy	9	41	0.711	0.710	0.711	0.710
## 11	entropy	9	32	0.711	0.711	0.711	0.710
## 12	entropy	8	26	0.711	0.710	0.711	0.710
## 13	entropy	9	40	0.711	0.710	0.711	0.710
## 14	entropy	8	25	0.710	0.710	0.710	0.710

A continuación entonces se visualiza la estabilidad de los 15 primeros árboles mediante boxplots de la validación cruzada.

```
cols = ['split0_test_accuracy', 'split1_test_accuracy', 'split2_test_accuracy', 'split3_test_accuracy']
res_df = results2.loc[:, cols]
data = [row.values for _, row in res_df.iterrows()]
labels = [f"res_{i}" for i in range(len(res_df))]
plt.figure(figsize=(12,6))
plt.boxplot(data, labels=labels)
plt.show()
```



El rango de variabilidad en los 15 modelos con mayor puntuación no es excesivamente grande pero no despreciable. A priori podrían ser más interesantes, por su robustez, los árboles de los índices 1,3,4,5 y 6, dado el tamaño de la caja. No obstante, con el código siguiente podemos extraer las puntuaciones AUC de los 15 árboles para considerarlos sobre los datos de test:

```
for i in range(15): # de 0 a 14
    params = results2['params'].iloc[i]
    best_model = DecisionTreeClassifier(**params)
    best_model.fit(X_train, y_train)
    y_train_pred = best_model.predict(X_train)
    y_test_pred = best_model.predict(X_test)
    y_train_auc = pd.get_dummies(y_train, drop_first=True)
    y_prob_train = best_model.predict_proba(X_train)[: , 1]
    fpr, tpr, thresholds = roc_curve(y_train_auc, y_prob_train)
    roc_auc_train = auc(fpr, tpr)
    print(f"\nValor AUC {i} (train): {roc_auc_train:.2f}")
    y_test_auc = pd.get_dummies(y_test, drop_first=True)
```

```

y_prob_test = best_model.predict_proba(X_test)[: , 1]
fpr, tpr, thresholds = roc_curve(y_test_auc, y_prob_test)
roc_auc_test = auc(fpr, tpr)
print(f"Valor AUC {i} (test): {roc_auc_test:.2f}")

## Valor AUC 0 (train): 0.89
## Valor AUC 0 (test): 0.81
## Valor AUC 1 (train): 0.88
## Valor AUC 1 (test): 0.81
## Valor AUC 2 (train): 0.89
## Valor AUC 2 (test): 0.80
## Valor AUC 3 (train): 0.88
## Valor AUC 3 (test): 0.81
## Valor AUC 4 (train): 0.89
## Valor AUC 4 (test): 0.81
## Valor AUC 5 (train): 0.88
## Valor AUC 5 (test): 0.81
## Valor AUC 6 (train): 0.88
## Valor AUC 6 (test): 0.81
## Valor AUC 7 (train): 0.89
## Valor AUC 7 (test): 0.80
## Valor AUC 8 (train): 0.89
## Valor AUC 8 (test): 0.80
## Valor AUC 9 (train): 0.89
## Valor AUC 9 (test): 0.80
## Valor AUC 10 (train): 0.87
## Valor AUC 10 (test): 0.82
## Valor AUC 11 (train): 0.88
## Valor AUC 11 (test): 0.81
## Valor AUC 12 (train): 0.88
## Valor AUC 12 (test): 0.81
## Valor AUC 13 (train): 0.88
## Valor AUC 13 (test): 0.82
## Valor AUC 14 (train): 0.87
## Valor AUC 14 (test): 0.82

```

Como se puede ver de estos resultados, tienen un comportamiento similar en las medidas de validación con los test, teniendo valores de AUC relativamente altos y sobre todo, con poca diferencia entre train y test. Dadas las similitudes entre las métricas, se elige el árbol de la posición 3, en función de las 4 métricas, la consistencia en la validación cruzada y la validación con los datos de test. En la sección 2.5 se trazan las áreas bajo la curva para las secciones de entrenamiento y de validación. Este árbol tiene unos datos del área bajo la curva de 0.81 y 0.88 respectivamente.

2.2 Reglas del árbol

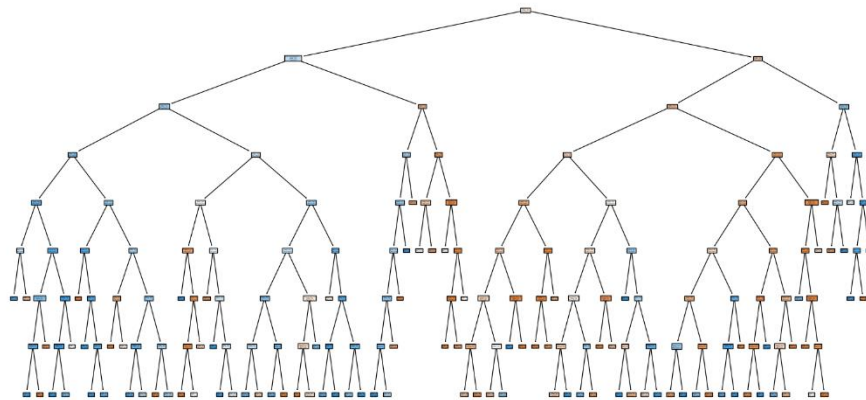
El código para reflejar las reglas del árbol en formato texto es el siguiente, con el árbol escogido (índice 3). Como acapara una cantidad de espacio inasumible en este ejercicio, porque es un árbol muy profundo, no se ejecuta el mismo. También se comprueban el accuracy en train y test para estimar el sobreajuste.

```

params = results2['params'].iloc[3]
best_model = DecisionTreeClassifier(**params)
print(f'Se tiene un accuracy para train de: {accuracy_score(y_train,y_train_pred)}')
0.800979262672811
print(f'Se tiene un accuracy para test de: {accuracy_score(y_test,y_test_pred)}')
0.7361751152073732
tree_rules = export_text(best_model, feature_names=list(X_train.columns),show_weights=True)
print(tree_rules)

```

Por lo tanto, se modifica el código original para que se muestre una proporción de las reglas del mismo:



```
## | | | | --- Price > 48455.00
## | | | | | --- weights: [4.00, 1.00] class: Black
## | | | | --- Levy > 901.00
## | | | | | --- Airbags <= 8.50
## | | | | | --- truncated branch of depth 2
## | | | | | --- Airbags > 8.50
## | | | | | --- truncated branch of depth 4
## | --- Engine volume > 2.30
## | | --- Levy <= 1892.50
## | | | --- Engine volume <= 3.35
## | | | | --- Levy <= 608.00
## | | | | | --- truncated branch of depth 5
## | | | | | --- Levy > 608.00
## | | | | | --- truncated branch of depth 5
```

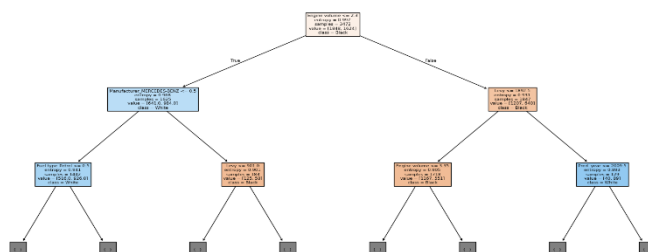
2.4 Visualización del árbol

El árbol entero se puede visualizar con este código.

```
plt.figure(figsize=(20,10))
plot_tree(best_model, feature_names=list(X_train.columns), class_names=best_model.classes_,
filled=True)
plt.show()
```

Y un detalle de los nodos superiores puede verse igualmente:

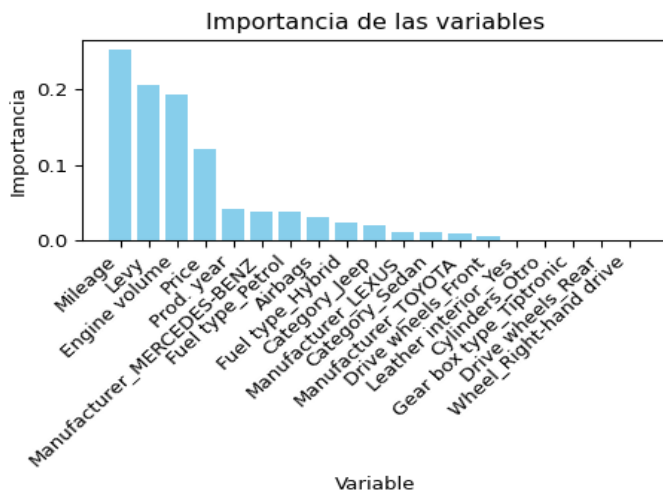
```
plt.figure(figsize=(20,10))
plot_tree(best_model, feature_names=list(X_train.columns), class_names=best_model.classes_,
filled=True,max_depth=2)
plt.show()
```



2.4 Importancia de las variables

Según el gráfico de barras creado, según ese modelo las variables cuantitativas explican una proporción mucho más importante que las cualitativas dummificadas del dataset cars.

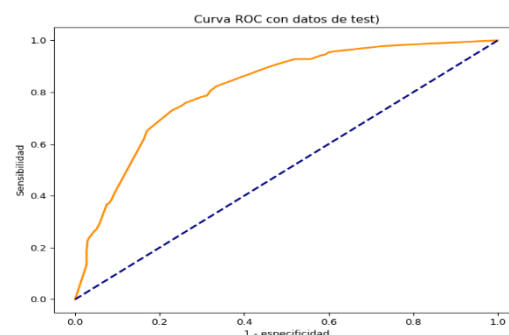
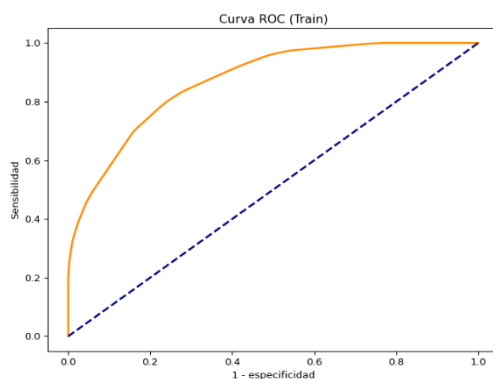
```
df_importancia = pd.DataFrame({'Variable': best_model.feature_names_in_, 'Importancia':  
best_model.feature_importances_}).sort_values(by='Importancia', ascending=False)  
plt.bar(df_importancia['Variable'], df_importancia['Importancia'], color='skyblue')  
plt.xlabel('Variable')  
plt.ylabel('Importancia')  
plt.title('Importancia de las variables')  
plt.xticks(rotation=45, ha='right')  
  
plt.tight_layout()  
plt.show()
```



2.5 Curvas de evaluación del modelo elegido

Finalmente queda por describir la curva ROC del modelo elegido, que tiene los valores para AUC de test y train de 0.81 y 0.88.

```
y_train_auc = pd.get_dummies(y_train, drop_first=True)  
y_prob_train = best_model.predict_proba(X_train)[:, 1]  
fpr, tpr, thresholds = roc_curve(y_train_auc, y_prob_train)  
roc_auc = auc(fpr, tpr)  
plt.figure(figsize=(8, 6))  
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'AUC = {roc_auc:.2f}')  
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')  
plt.xlabel('1 - especificidad')  
plt.ylabel('Sensibilidad')  
plt.title('Curva ROC (Train)')  
plt.show()
```



3. Mejor búsqueda paramétrica por Random Forest y XGBoost

3.1 Búsqueda paramétrica por Random Forest

El primer modelo inicial de Random Forest ya nos da mejores resultados de precisión, siempre con los datos de train:

```
from sklearn.ensemble import RandomForestRegressor, RandomForestClassifier
RF_model = RandomForestClassifier(n_estimators = 60, bootstrap = True, max_depth = 8,
min_samples_split=29, criterion='entropy', random_state=1234)
RF_model.fit(X_train, y_train)
## RandomForestClassifier(criterion='entropy', max_depth=8, min_samples_split=29,
## n_estimators=60, random_state=1234)
y_pred_rf = RF_model.predict(X_test)
accuracy_rf = accuracy_score(y_test, y_pred_rf)
print(f'Precisión del modelo con Random Forest: {accuracy_rf}')
## Precisión del modelo con Random Forest: 0.7764976958525346
```

Para este apartado se trabaja con una malla de hiperparámetros bastante amplia de partida, pero basada en los parámetros seleccionados del árbol de clasificación anterior (eg. Profundidad).. Como en el caso de la búsqueda del árbol de clasificación, no se considera el tamaño mínimo de hoja.

```
params = {
    'n_estimators': [50, 100, 200, 300, 350],
    'max_depth': list(range(6, 9)),
    'bootstrap': [True, False],
    'min_samples_split': list(range(25, 75)),
    'criterion': ["gini", "entropy"]
}
scoring_metrics = ['accuracy', 'precision_macro', 'recall_macro', 'f1_macro']
grid_search_RF = GridSearchCV(estimator=RF_model,
                              param_grid=params,
                              cv=4, scoring = scoring_metrics, refit='accuracy')
grid_search_RF.fit(X_train, y_train)
## GridSearchCV(cv=4,
## estimator=RandomForestClassifier(criterion='entropy', max_depth=8,
## min_samples_split=29,
## n_estimators=60,
## random_state=1234),
## param_grid={'bootstrap': [True, False],
## 'criterion': ['gini', 'entropy'],
## 'max_depth': [6, 7, 8],
## 'min_samples_split': [25, 26, 27, 28, 29, 30, 31, 32,
## 33, 34, 35, 36, 37, 38, 39, 40,
## 41, 42, 43, 44, 45, 46, 47, 48,
## 49, 50, 51, 52, 53, 54, ...],
## 'n_estimators': [50, 100, 200, 300, 350]},
## refit='accuracy',
## scoring=['accuracy', 'precision_macro', 'recall_macro',
## 'f1_macro'])
best_model_RF = grid_search_RF.best_estimator_
print(grid_search_RF.best_estimator_)
## RandomForestClassifier(bootstrap=False, max_depth=8, min_samples_split=27,
## random_state=1234)
y_pred_rf = best_model_RF.predict(X_test)
accuracy_rf_gs = accuracy_score(y_test, y_pred_rf)
print(f'Precisión del árbol optimizado: {accuracy_rf_gs}')
## Precisión del árbol optimizado: 0.7799539170506913
```

La estimación de la calidad del mejor modelo según la función `best_estimator` sería la siguiente. No se puede afirmar que haya sobre ajuste, pues hay poca diferencia en la estimación de la precisión.

```

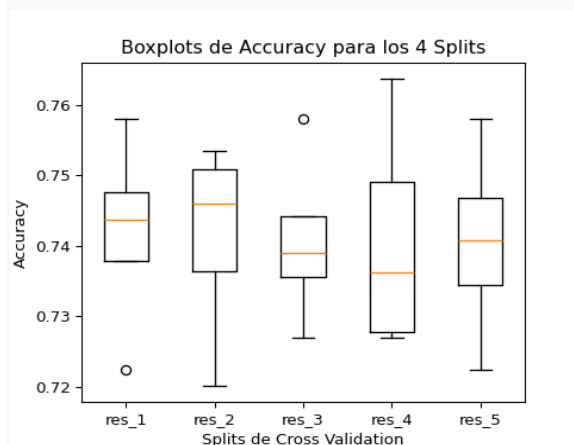
y_train_pred = best_model_RF.predict(X_train)
y_test_pred = best_model_RF.predict(X_test)
print(f'Se tiene un accuracy para train de: {accuracy_score(y_train,y_train_pred)}')
## Se tiene un accuracy para train de: 0.8070276497695853
print(f'Se tiene un accuracy para test de: {accuracy_score(y_test,y_test_pred)}')
## Se tiene un accuracy para test de: 0.7799539170506913

```

Si se ordenan los datos de métricas de precisión, se obtienen muchas opciones muy similares en cuanto a las medidas de bondad, que son nuevamente consistentes entre las cuatro. Ahora se van a tomar las 5 primeras y se va a observar la robustez a lo largo de las cuatro pruebas de validación cruzada. El árbol seleccionado en este caso será el primero.

##	bootstrap	criterion	depth	split	n_estimators	accu	precision	recall	\
## 0	False	gini	8	27	100	0.742	0.741	0.742	
## 5	False	gini	8	30	100	0.740	0.740	0.740	
## 10	False	gini	8	25	200	0.739	0.738	0.738	
## 15	False	gini	8	30	200	0.738	0.738	0.738	
## 20	False	gini	8	29	100	0.738	0.737	0.737	
## 25	False	entropy	8	26	300	0.738	0.737	0.737	
## 30	False	entropy	8	26	200	0.737	0.737	0.737	

##	bootstrap	criterion	depth	split	n_estimators	accu	precision	recall	\
## 0	False	gini	8	27	100	0.742	0.741	0.742	
## 1	False	gini	8	35	100	0.741	0.741	0.741	
## 2	False	gini	8	26	50	0.741	0.740	0.741	
## 3	False	entropy	8	27	200	0.741	0.740	0.741	
## 4	False	gini	8	28	100	0.740	0.740	0.740	
## 5	False	gini	8	30	100	0.740	0.740	0.740	
## 6	False	gini	8	28	200	0.740	0.739	0.740	
## 7	False	entropy	8	28	200	0.739	0.739	0.739	
## 8	False	gini	8	25	100	0.739	0.739	0.739	
## 9	False	entropy	8	26	100	0.739	0.738	0.738	
## 10	False	gini	8	25	200	0.739	0.738	0.738	
## 11	False	gini	8	34	100	0.739	0.738	0.739	
## 12	False	gini	8	29	200	0.739	0.738	0.739	
## 13	False	entropy	8	25	100	0.739	0.738	0.738	
## 14	False	gini	8	27	350	0.738	0.738	0.738	



En este caso podría decirse que los modelos más robustos son el primero y el tercero (por alguna razón aquí Python se saltó el índice 0). Cogiendo el valor de accuracy se aceptaría el primer modelo. Para el árbol de Random Forest, hay valores no muy alejados entre train y test para los datos de AUC, indicando tanto un buen comportamiento del modelo como la falta de sobreajuste.

```

y_train_auc = pd.get_dummies(y_train, drop_first=True)
y_prob_train = best_model.predict_proba(X_train)[: , 1]
fpr, tpr, thresholds = roc_curve(y_train_auc, y_prob_train)

```

```

roc_auc_train = auc(fpr, tpr)
print(f"\nValor AUC (train): {roc_auc_train:.2f}")
## Valor AUC (train): 0.87
y_test_auc = pd.get_dummies(y_test, drop_first=True)
y_prob_test = best_model.predict_proba(X_test)[: , 1]
fpr, tpr, thresholds = roc_curve(y_test_auc, y_prob_test)
roc_auc_test = auc(fpr, tpr)
print(f"Valor AUC (test): {roc_auc_test:.2f}")
## Valor AUC (test): 0.82

```

3.2 Optimización con XGBoost

La optimización con XGBoost también partirá de un clasificador de base.

```

import xgboost as xgb
y_trainB = y_train.map({'White': 0, 'Black': 1})
y_testB = y_test.map({'White': 0, 'Black': 1})
xgb_classifier = xgb.XGBClassifier(booster = 'gbtree', n_estimators = 400,
                                  eta = 0.1, gamma = 1, random_state=1234, max_depth = 9, tree_method =
'hist')
xgb_classifier.fit(X_train, y_trainB)
## XGBClassifier(base_score=None, booster='gbtree', callbacks=None,
##               colsample_bylevel=None, colsample_bynode=None,
##               colsample_bytree=None, device=None, early_stopping_rounds=None,
##               enable_categorical=False, eta=0.1, eval_metric=None,
##               feature_types=None, feature_weights=None, gamma=1,
##               grow_policy=None, importance_type=None,
##               interaction_constraints=None, learning_rate=None, max_bin=None,
##               max_cat_threshold=None, max_cat_to_onehot=None,
##               max_delta_step=None, max_depth=9, max_leaves=None,
##               min_child_weight=None, missing=nan, monotone_constraints=None,
##               multi_strategy=None, n_estimators=400, n_jobs=None, ...)
y_pred_baseB = xgb_classifier.predict(X_test)
# Evaluar el rendimiento del modelo
accuracy_a = accuracy_score(y_testB, y_pred_baseB)
print(f'Precisión de gradient boosting: {accuracy_a}')
## Precisión de gradient boosting: 0.7811059907834101
params = {
    'n_estimators': [100,200,300,400],
    'eta' : [0.1,0.25,0.5,0.7],
    'gamma' : [0.1,0.25,0.5,0.75,1],
    'max_depth': [5,6,7,8,9]
}
scoring_metrics = ['accuracy', 'precision_macro', 'recall_macro', 'f1_macro']
grid_search_XGB = GridSearchCV(estimator=xgb_classifier, param_grid=params,
                               cv=4, scoring = scoring_metrics, refit='accuracy')
grid_search_XGB.fit(X_train, y_trainB)

```

Los parámetros obtenidos se presentan a continuación. Tasas de aprendizaje altas (eta) en general para los que puntúan mayor, menos profundidad del árbol y valores de accuracy de 0.771, los más altos, al igual que el resto de medidas. Los 5 primeros árboles son prácticamente idénticos en la robustez por validación cruzada y las cuatro métricas de bondad. La profundidad varía entre 8 y 9. Para intentar no sobreajustar el modelo se selecciona el primer modelo, aunque es peor en promedio que el quinto según el boxplot, tiene algo menos de profundidad.

```

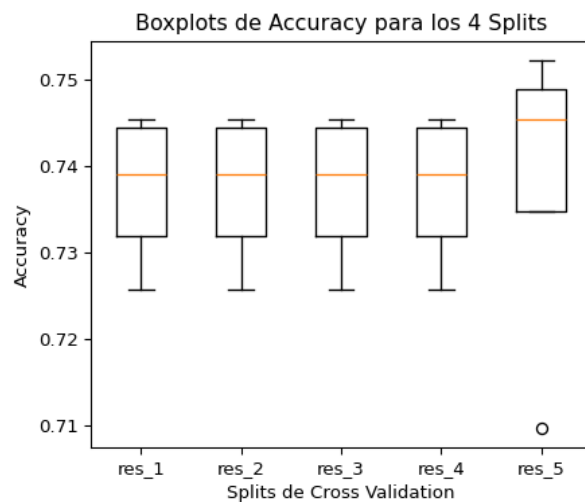
print(grid_search_XGB.best_estimator_.get_params())
## <bound method XGBModel.get_params of XGBClassifier(base_score=None, booster='gbtree',
## callbacks=None,
##               colsample_bylevel=None, colsample_bynode=None,
##               colsample_bytree=None, device=None, early_stopping_rounds=None,
##               enable_categorical=False, eta=0.7, eval_metric=None,
##               feature_types=None, feature_weights=None, gamma=0.1,

```



```
##          grow_policy=None, importance_type=None,
##          interaction_constraints=None, learning_rate=None, max_bin=None,
##          max_cat_threshold=None, max_cat_to_onehot=None,
##          max_delta_step=None, max_depth=8, max_leaves=None,
##          min_child_weight=None, missing=nan, monotone_constraints=None,
##          multi_strategy=None, n_estimators=100, n_jobs=None, ...)>
results_abrevXGB.head(15)
```

	eta	gamma	depth	n_estimators	acc	precision	recall	f1
## 0	0.70	0.1	8	300	0.771	0.770	0.769	0.770
## 1	0.70	0.1	8	400	0.771	0.770	0.769	0.770
## 2	0.70	0.1	8	200	0.771	0.770	0.769	0.770
## 3	0.70	0.1	8	100	0.771	0.770	0.769	0.770
## 4	0.10	0.1	9	100	0.767	0.766	0.766	0.766
## 5	0.25	0.1	9	300	0.767	0.766	0.765	0.765
## 6	0.25	0.1	9	200	0.767	0.766	0.765	0.765
## 7	0.25	0.1	9	100	0.767	0.766	0.765	0.765
## 8	0.25	0.1	9	400	0.767	0.766	0.765	0.765
## 9	0.50	0.1	6	200	0.766	0.766	0.765	0.765
## 10	0.50	0.1	6	300	0.766	0.766	0.765	0.765



```
modelo_XGB = grid_search_XGB.best_estimator_
y_train_pred_xgbB = modelo_XGB.predict(X_train)
y_test_pred_xgbB = modelo_XGB.predict(X_test)
print(f'Se tiene un accuracy para train de: {accuracy_score(y_trainB,y_train_pred_xgbB)}')
## Se tiene un accuracy para train de: 0.9959677419354839
print(f'Se tiene un accuracy para test de: {accuracy_score(y_testB,y_test_pred_xgbB)}')
## Se tiene un accuracy para test de: 0.7788018433179723
```

En el caso se obtienen dos datos muy llamativos: una diferencia muy importante que puede indicar un sobreajuste, dada el accuracy de casi 100%, muy sospechoso. El segundo dato también apunta al sobreajuste, por la diferencia que hay entre train y test. Los valores de bondad de AUC apuntan a lo mismo: un AUC absolutamente perfecto de 1, pero que disminuye hasta 0.87, que sigue siendo un buen indicador del comportamiento del modelo pero que no evita el sobreajuste.

```
y_train_aucB = pd.get_dummies(y_trainB, drop_first=True)
y_prob_trainB = modelo_XGB.predict_proba(X_train)[: , 1]
fpr, tpr, thresholds = roc_curve(y_train_aucB, y_prob_trainB)
roc_auc_train = auc(fpr, tpr)
print(f'\nValor AUC (train): {roc_auc_train:.2f}')
## Valor AUC (train): 1.00
y_test_aucB = pd.get_dummies(y_testB, drop_first=True)
y_prob_testB = modelo_XGB.predict_proba(X_test)[: , 1]
fpr, tpr, thresholds = roc_curve(y_test_aucB, y_prob_testB)
```

```
roc_auc_test = auc(fpr, tpr)
print(f"Valor AUC (test Boosting): {roc_auc_test:.2f}")
## Valor AUC (test Boosting): 0.87
```

4. Comparativa de los modelos:

En este ejercicio se ha realizado el desarrollo de tres modelos de clasificación basados en árboles sobre un dataset depurado para la ocasión, que podría admitir un mayor número de transformaciones.

Se observan las siguientes comparativas de los modelos elegidos. No se incluyen otras métricas de bondad

	Depth	Accuracy train	Accuracy test	AUC train	AUC test
Árbol	9	0.80	0.74	0.88	0.81
Random Forest	8	0.81	0.78	0.87	0.82
XGBoost	8	1	0.78	1.00	0.87

Paulatinamente se han ido encontrando mejores métricas de bondad con los algoritmos de aprendizaje que con los árboles. Particularmente, XGBoost logra además mejores métricas que Random Forest, pero a costa de un mayor sobreajuste. Aunque llega a una precisión perfecta a la hora de clasificar el dataset de entrenamiento, los valores son sensiblemente inferiores para los datos de test. A pesar de ello se trata de un modelo que clasifica razonablemente bien según los datos de test, y según el problema planteado de análisis de datos y predicción, puede funcionar bastante bien.

Los modelos coinciden en los rangos de profundidad del árbol y allí donde se puede medir (Random Forest y árbol de clasificación), también hay valores similares en los tamaños de nodo entre los algoritmos. Es decir, la consistencia entre los modelos es razonablemente buena. Por otra parte, a efectos de la comparativa de este apartado de la tarea, existen muchos parámetros que por extensión no pueden ser tenidos en cuenta o bien no existen en otros algoritmos. Con la panorámica de todas las métricas, se puede concluir de que la técnica que mejor funciona es Random Forest con esta calibración, puesto que alcanza unas métricas bastante buenas y sobreajusta menos. Por otra parte, ha prevalecido el criterio de entropía frente al coeficiente de Gini a la hora de realizar los cálculos de los nodos.

Otras consideraciones:

- Según los objetivos que tiene construir un modelo, se debe trabajar con más o menos riesgo al sobreajuste en detrimento de la precisión, especialmente cuando los tamaños muestrales, como en este caso, son pequeños. Se ha mantenido cierta atención al riesgo de sobreajuste, sobre todo viendo cómo con selecciones previas de modelos automáticas, no incluidas, el algoritmo tiende a elegir árboles mucho más profundos, que llevan a discrepancias de casi el 30% en las métricas de bondad entre entrenamiento y test. La segunda es que se ha trabajado intentando seguir una lógica de mejora respecto al algoritmo de árbol de clasificación más

simple, que se ha intentando calibrar correctamente en la medida de lo posible. Se ha mantenido la base de profundidad del árbol de partida para realizar las clasificaciones. De los resultados de calidad extraigo los siguientes puntos:

- Un virtud extremadamente útil desde el punto de vista de la interpretación del modelo es la importancia de las variables, pues puede dar mucha información para el uso commercial que se le dé posteriormente a los datos, o la gestion.
- La búsqueda de hiperparámetros necesita acotarse de alguna manera para encontrar un equilibrio entre la calidad del modelo y el tiempo computacional y del analista que esté con estos datos. Además de las decisiones y expectativas de empresa, se necesita un conocimiento profundo para entender las implicaciones de cada hiperparámetro de cara a la interpretación.
- La última consideración es mi experiencia profesional previa aplicando árboles en mi sector (biología vegetal y de la conservación), donde se aplican, con paquetes ad hoc de R, combinaciones de técnicas que incluyen tanto GLMs y GAMs, como Random Forest, CART, GBM etc., para predecir las áreas donde una determinada especie puede estar o no, basándose por ejemplo en predictores climáticos. Se están publicando artículos que incorporan esta aproximación basándose entre otras cosas en parametrizaciones automáticas de los árboles y tamaños muestrales en muchos casos de $n > 100$ y con evaluaciones exclusivamente sobre datos de entrenamiento; quiero resaltar por lo tanto la utilidad de este módulo para la optimización mucho más ajustada de las técnicas de Machine Learning.